

Team members:

Andrej Ermoshkin - Part C

Fortune Meya - Part D

Ashley Celis - Assignment report and part E

- Chord integration
 - The DFS uses chord to assign file metadata and content pages to nodes, ensuring that a file's metadata will map to its successor on the identifier ring.
- Metadata and page design
 - Files are decomposed into smaller pages. The file metadata, the name, size, permissions, pagemapping, is stored as a key-value pair, where the key is hash("metadata" + filename). The actual file content is split into pages. Every page has a unique identifier, and the metadata maps which pages belong to which file. The metadata object contains the filename, the size in bytes, the number of pages, and the version number. Each descriptor contains the page number, a GUID, and the replicas. The page object stores a chunk of the file content and it is placed in the chord ring using hash(filename + ":" + page_number), which routes to the node responsible for the successor.
- Distributed sorting strategy
 - To sort large files in a distributed manner, the DFS routes individual records across their peers based on the key hash. First, each page of the input file is parsed into key-value record pairs. For every record, the sort key is hashed into the chord ring and the responsible peer is identified. Then, the record is sent to that peer, where it is inserted into a local ordered structure. Specifically, a sorted list that has ascending key order on every insert. When all records are routed, the sorted output is created using the locally sorted records from each peer in ring order and performing a merge pass. The output file is validated by reading all records and asserting non-decreasing key order. This ensures that each

peer is responsible for a range of keys, and the sorted state is maintained as records arrive without requiring a separate sort step.

- Replication strategy
 - To improve fault tolerance, each file page is replicated to the successors of the node responsible for the key in the Chord ring, which ensures that the data survives node departures.
- Paxos message flow
 - To make sure that all replicas receive updates in the same order, a simplified Paxos protocol is used. The leader will propose an operation by generating a ballot number sending an ACCEPT message to the replicas. The replicas check that the ballot is the highest it has seen. If it is, the operation is recorded and it responds with a LEARN message to the leader. When the leader receives this response from most replicas, the operation is committed. All replicas will then apply the operation to their local store in the same order, which is guaranteed by the increasing ballot numbers.
- Failure assumptions
 - The system remains connected using Chord's stabilization protocol. If a node were to fail, the replication factor ensures that data is found on the successor node. Paxos handles message delays or losses, as the algorithm does not rely on timely delivery, it is only on a majority quorum of acceptors to make progress.
- Limitations and future improvements
 - The system performance depends on the stability of the chord ring. A simplified Paxos approach may increase latency due to multiple message rounds per write.

Test Evidence:

- Sample input file
this is a test
this project is for the cecs 327 class

we are storing pages over chord

last line here

- Sample output file

last line here

this is a test

this project is for the cecs 327 class

we are storing pages over chord

- Screenshots or logs of Paxos messages

[PAXOS][replica1] ACCEPT(3, delete_file)

[PAXOS][replica2] ACCEPT(3, delete_file)

[PAXOS][replica3] ACCEPT(3, delete_file)

[PAXOS][replica1] LEARN(3, delete_file)

[PAXOS][replica2] LEARN(3, delete_file)

[PAXOS][replica3] LEARN(3, delete_file)

[PAXOS][LEADER=replica1] COMMIT(3, delete_file)

FILES AFTER DELETE: []

- Correctness checks

- Read original file:

this is a test

this project is for the cecs 327 class

we are storing pages over chord

last line here

- Sort original file:

last line here

this is a test

this project is for the cecs 327 class

we are storing pages over chord

- Leader receives LEARN messages from replicas

[PAXOS][replica1] ACCEPT(2, append)

[PAXOS][replica2] ACCEPT(2, append)

[PAXOS][replica3] ACCEPT(2, append)

[PAXOS][replica1] LEARN(2, append)

[PAXOS][replica2] LEARN(2, append)

[PAXOS][replica3] LEARN(2, append)

[PAXOS][LEADER=replica1] COMMIT(2, append)

FILES: ['report.txt']